Crabbity Crab 🦀

Product Requirement Document (PRD) v1

Logistics · Scenario 2: Warehouse Stock Bling Stocks

Problem context: Small-to-mid warehouses don't know what they have until it's too late (overstock or stockout). Manual inventory counts are slow and error-prone. There is no forecasting: recording is reactive, not predictive. System records and physical stock often mismatch.

Document Status	Confirmed Concept by Liam
Product Type	Mobile Application
Target Market	Small businesses and convenience stores
Initial Platform	Mobile
Initial Users	One user managing one store
MVP Development Timeline	5 days

```
The current product demo name is referred to as "Smart Stock" in this PRD.
```

1. Product Overview

1.1 What Is Smart Stock?

Smart Stock is a mobile-first inventory management application designed to help small business owners manage product stock, track inventory changes, identify stock shortages, and maintain more accurate inventory records.

Instead of relying on manual calculations, paper records, or spreadsheets, store owners can use their mobile phones to register products, record stock movements, conduct physical inventory counts, and record sales.

The application focuses on making inventory management simple, practical, and accessible to people who do not have dedicated inventory management systems. Its goal is to help users understand their inventory situation and take action before inventory problems affect their business.

1.2 Product Vision

Help small business owners maintain accurate inventory, reduce stock discrepancies, and make better stocking decisions through a simple, mobile-first inventory management experience.

2. Problem Statement

Small business owners often manage inventory manually or use tools that do not adequately support their daily operations. This creates several problems.

2.1 Inaccurate Inventory Records

Products may be sold, received, damaged, or removed without the inventory records being updated correctly. The quantity displayed in the system may not match the actual quantity available in the store.

2.2 Time-Consuming Stock Counting

Manually counting products and updating records can be tedious, particularly when a store carries hundreds of products.

2.3 Unexpected Stockouts

Store owners may not realize that a product is running low until customers attempt to purchase it.

2.4 Forgotten Transactions

A store owner may forget to record a sale or stock movement during a busy period. The system must account for the possibility of missing transactions instead of assuming every inventory change will always be recorded.

2.5 Limited Inventory Visibility

Basic inventory tools may display product quantities without clearly identifying which products require attention. Store owners need a practical way to identify low-stock items, investigate discrepancies, and decide what to do next.

3. Target Audience

3.1 Primary Users

Small business owners who manage their own product inventory, such as convenience store owners, small retail shop owners, independent shopkeepers, and small businesses with approximately 200 products.

3.2 Initial User Scenario

The initial version is designed around one store owner managing one convenience store. The owner uses a mobile phone to:

- Register and manage products.
- Record new stock arriving at the store.
- Record stock leaving the inventory.
- Record product sales through a lightweight checkout workflow.
- Count physical stock and correct discrepancies.
- Review inventory changes.
- Identify products that require restocking.

3.3 Initial Operating Environment

The MVP assumes one store, one combined store inventory, approximately 200 products as the initial planning assumption, one user managing the inventory, and mobile-first daily operations. The initial version does not require multiple warehouses, separate storage locations, or a team of employees.

4. Product Goals

The MVP has five primary goals.

1. **Simplify Inventory Management.** Allow users to manage products and stock movements directly from their mobile phones.
2. **Improve Inventory Accuracy.** Provide a physical counting and stock adjustment workflow so users can correct differences between recorded quantities and actual stock.
3. **Reduce Unexpected Stockouts.** Identify products with low or zero stock and make those conditions visible to users.
4. **Make Inventory Changes Traceable.** Maintain a history of stock movements and adjustments so users can understand why inventory quantities changed.
5. **Keep the Product Simple and Scalable.** Build a focused MVP that solves the core inventory problem while leaving room for future team collaboration and multi-location support.

5. Core Features

5.1 Product Management

Users need a central catalog of products available in their store.

Required capabilities

- View the product inventory.
- Search for products.
- Register new products manually.
- Enter product information and initial stock quantity.
- View current stock quantities.
- Identify low-stock and out-of-stock products.

Product identification. Not every product has a barcode or QR code, so manual product registration and search must remain available. Barcode or QR scanning may be supported as an optional shortcut, but scanning must not be mandatory for managing inventory.

> ***Confirmed decision:*** *The application must work for products without barcodes or QR codes.*

The exact product fields can be finalized during interface and implementation design.

5.2 Stock In

Stock In records products entering the store's inventory.

Example scenario. A supplier delivers 20 bottles of water. The user opens the inventory, selects or searches for the water product, enters the incoming quantity of 20, and confirms the stock movement. The system increases the recorded quantity by 20 and records the movement in the inventory history.

Expected result. The inventory reflects the newly received stock, and the movement can be reviewed later.

5.3 Stock Out

Stock Out records products leaving inventory for reasons other than a sale, such as damaged, expired, or lost products, or other recorded inventory removals.

Example scenario. A store owner discovers that three products are damaged. The owner selects the product, enters a quantity of three, chooses the appropriate reason when supported, and confirms the movement. The system reduces the recorded quantity and maintains a record of the change.

Sales should be handled through the separate lightweight checkout workflow rather than requiring every sale to be entered as a generic stock-out movement.

5.4 Physical Stock Counting

Physical stock counting allows users to verify how many units are actually available in the store. This is particularly important because real-world inventory records can become inaccurate when sales or other stock movements are missed.

Example scenario. The system indicates that 15 units of a product should remain. The owner physically counts the shelf and storage area and discovers that only 12 units remain. The owner enters the actual quantity of 12. The system identifies the discrepancy and allows the owner to reconcile the inventory.

Expected result. The system updates the recorded stock to match the confirmed physical quantity. The adjustment should be recorded in the inventory history.

Important distinction. Physical stock counting is not the same as recording a normal stock-out transaction. A stock-out records a known movement. A physical count verifies the quantity that actually exists, including the effects of movements that may not have been recorded previously. This makes physical counting an essential recovery mechanism for maintaining inventory accuracy.

5.5 Inventory Adjustments and History

Inventory adjustments allow users to correct discrepancies discovered during physical stock counting.

Required capabilities

- Record the previous stock quantity.
- Record the corrected quantity.
- Calculate the difference.
- Record the adjustment.
- Display inventory movement and adjustment history.

Detail	Quantity
Previously recorded stock	15
Actual physical stock	12
Adjustment	-3
Updated stock	12

The history should help users understand what changed rather than silently replacing the previous quantity. The precise history fields and filtering options can be finalized during implementation.

5.6 Low-Stock and Out-of-Stock Alerts

The system should help users identify products that require attention.

- **Low-stock alert:** a product is flagged when its recorded quantity reaches its configured low-stock threshold.
- **Out-of-stock alert:** a product is flagged when its recorded quantity reaches zero.

Expected capabilities: identify low-stock products, identify out-of-stock products, display inventory conditions clearly, and allow users to find the affected products quickly.

The MVP should prioritize useful inventory visibility over complex forecasting or automated purchasing. The exact threshold configuration, notification channels, and alert delivery behavior remain implementation details to finalize.

5.7 Lightweight Checkout

The MVP includes a simple checkout workflow to record product sales and automatically deduct the sold quantities from inventory. This is not intended to be a complete point-of-sale system. Its primary purpose is to connect sales records with inventory changes.

Checkout workflow

1. The user opens checkout.
2. The user searches for or selects a product.
3. The user enters the quantity sold.
4. The user adds the item to the sale.
5. The user reviews the items and quantities.
6. The user completes the sale.
7. The system deducts the sold quantities from inventory.
8. The system records the corresponding inventory changes.

Example scenario. A customer purchases five bottles of water. Before the sale, the recorded inventory is 30 bottles. The owner records five bottles in checkout and completes the sale. The system updates the recorded inventory: 30 − 5 = 25 bottles. The user does not need to scan every individual bottle or item; products can be selected through search, and quantities can be entered manually.

Payment processing. Actual payment processing is not required for the MVP. The application records that a sale has been completed for inventory purposes. It does not need to process card payments, mobile payments, or other financial transactions.

Why checkout is included. Without a sales-recording workflow, inventory would become inaccurate whenever products were sold but no corresponding stock movement was recorded. Lightweight checkout provides a simple way to reduce this problem. However, the application must not assume that every sale will always be recorded. Physical stock counting and inventory adjustments remain necessary to correct discrepancies caused by forgotten sales or other unrecorded movements.

6. Inventory Management Logic

The following rules define the intended inventory behavior.

6.1 Initial Stock

When a product is registered, the user can enter its initial quantity. That quantity becomes the starting recorded stock.

6.2 Stock In

Updated Stock = Previous Stock + Incoming Quantity. Example: 50 + 20 = 70 units.

6.3 Stock Out

Updated Stock = Previous Stock − Removed Quantity. Example: 50 − 3 = 47 units.

6.4 Completed Sale

Updated Stock = Previous Stock − Sold Quantity. Example: 50 − 5 = 45 units.

The system should deduct stock only when the sale is successfully completed, not merely when the user selects a product or adds it to the checkout list.

6.5 Physical Count Adjustment

When the user confirms a physical stock count: Adjustment = Actual Quantity − Previously Recorded Quantity, and Updated Stock = Actual Quantity.

Example: previously recorded quantity 20, actual quantity 17, adjustment −3, updated quantity 17.

6.6 Inventory Consistency

All supported stock-changing actions must update the same store inventory. The product quantity displayed in the inventory should reflect completed stock movements, completed sales, and confirmed adjustments. A product must not be maintained as separate, disconnected quantities simply because it was updated through different features.

6.7 Missing or Forgotten Sales

If a sale is not recorded, the system cannot automatically know that the product has left the store. The discrepancy may be discovered during a physical stock count, and the user can then adjust the recorded quantity to match the actual quantity. This is an accepted limitation of the MVP.

7. User Experience Principles

- **7.1 Mobile First.** The initial experience is designed for mobile phones because the target user needs to manage stock during normal store operations.
- **7.2 Fast and Practical.** Common actions should require as few steps as reasonably possible. Users should be able to search for products and enter quantities without unnecessary complexity.
- **7.3 Manual Entry Always Available.** Barcode scanning must not be the only way to register, find, or manage products.
- **7.4 Clear Inventory Status.** Users should be able to distinguish normal stock, low stock, and out-of-stock conditions.
- **7.5 Transparent Adjustments.** The system should maintain a history of inventory changes rather than hiding corrections.
- **7.6 Smart but Not Complicated.** The product should surface actionable inventory information without overwhelming users with complex analytics or advanced inventory terminology.

8. MVP Scope

The MVP is the smallest practical version that demonstrates the core value of Smart Stock.

8.1 Included Features

Feature	MVP Status
Mobile application	Included
Single-store inventory	Included
Single-user operation	Included
Product registration	Included
Product search	Included
Manual product identification	Included
Optional barcode or QR scanning	Optional capability
Initial stock entry	Included
Stock In	Included
Stock Out	Included

Feature	MVP Status
Physical stock counting	Included
Inventory adjustments	Included
Inventory movement history	Included
Low-stock identification	Included
Out-of-stock identification	Included
Lightweight checkout	Included
Manual product and quantity selection during checkout	Included
Automatic inventory deduction after completed sale	Included
Actual payment processing	Not required
Multiple users	Excluded
Multiple warehouses	Excluded
Multiple store locations	Excluded

Optional barcode or QR scanning should be implemented only to the extent practical within the five-day timeline. Manual product registration and search are essential.

8.2 Explicitly Out of Scope

The MVP does not aim to provide:

- A complete point-of-sale system.
- Payment gateway integration.
- Multiple employee accounts.
- Team permissions or role management.
- Multi-warehouse inventory.
- Multi-location inventory management.
- Transfers between shelves, warehouses, or store locations.
- Advanced demand forecasting.
- Automated purchase orders.
- Complex business analytics.

These exclusions keep the first version focused on inventory accuracy and daily stock management.

9. Initial Application Structure

The application should provide a straightforward path to the core inventory workflows. The following is a proposed functional structure, not a final screen-by-screen design specification.

9.1 Inventory Overview

Purpose: Help users understand the current inventory situation. Potential content: total products, low-stock products, out-of-stock products, product search, inventory list, and shortcuts to common inventory actions. The overview should emphasize information that helps the user decide what needs attention.

9.2 Product List and Product Details

Purpose: Allow users to find products and inspect their inventory information. Users should be able to search products, view recorded quantities, identify stock status, open individual product details, and access relevant stock management actions.

9.3 Stock Movement

Purpose: Record incoming and outgoing inventory. Users should be able to select a product, enter the quantity, confirm the movement, and review the result.

9.4 Physical Stock Count

Purpose: Verify recorded quantities against actual stock. Users should be able to enter the actual quantity and review the resulting discrepancy before confirming an adjustment.

9.5 Checkout

Purpose: Record completed sales and update inventory. Users should be able to select multiple products, enter quantities, review the sale, and complete it without processing an actual payment.

9.6 Inventory History

Purpose: Make inventory changes traceable. Users should be able to review recorded movements and adjustments.

The final navigation structure, and whether these functions appear as separate screens or integrated workflows, will be determined during UI design.

10. Success Criteria

The MVP should be evaluated against the following criteria.

Product Management

- Users can register products manually.
- Users can find products through search.
- Products without barcodes can be managed.

Inventory Operations

- Stock In increases the recorded quantity correctly.
- Stock Out decreases the recorded quantity correctly.
- Completed sales decrease the recorded quantity correctly.
- Physical stock adjustments reconcile the recorded quantity with actual stock.

Traceability

- Stock movements and adjustments are recorded in history.
- Users can understand the difference between normal stock movements and physical count corrections.

Inventory Visibility

- Low-stock products can be identified.
- Out-of-stock products can be identified.
- Inventory status reflects the current recorded quantity.

Checkout

- Users can select products and enter quantities.
- Users can complete a sale without actual payment processing.
- Inventory is deducted when the sale is completed.

Usability

- The main workflows are practical on a mobile phone.
- Users can manage inventory without relying on barcode scanning.
- The application can demonstrate the core workflow within the initial development timeline.

11. Limitations and Risks

- **12.1 Unrecorded Transactions.** If users forget to record a sale or stock movement, the recorded quantity may become inaccurate. *Mitigation:* provide physical stock counting and adjustment workflows.
- **12.2 Incorrect Physical Counts.** The system relies on users entering accurate physical quantities. *Mitigation:* allow users to review discrepancies before confirming adjustments and preserve the adjustment history.
- **12.3 Scope Expansion.** Adding advanced checkout, payment processing, multi-user support, or multi-location management would increase complexity. *Mitigation:* keep these capabilities outside the MVP.
- **12.4 Optional Scanning Complexity.** Barcode or QR scanning may introduce implementation complexity and should not block the core inventory workflow. *Mitigation:* keep manual registration and search as the primary fallback and essential functionality.
- **12.5 Inventory Consistency.** If stock changes are not handled consistently across checkout, stock movements, and adjustments, product quantities may become incorrect. *Mitigation:* ensure every completed inventory-changing action updates the same underlying inventory quantity and produces the appropriate history record.

12. Future Roadmap

The initial version is intentionally limited to one store and one user. Future development may expand the product after the core inventory workflow has been validated.

Phase 1: Smart Stock MVP

One store, one user, mobile inventory management, product registration and search, Stock In and Stock Out, physical stock counting, inventory adjustments and history, low-stock and out-of-stock visibility, and lightweight checkout.

Phase 2: Team Collaboration

Potential capabilities: multiple users, employee accounts, shared inventory, role-based permissions, and activity attribution to individual users. The goal would be to support stores where several people manage the same inventory.

Phase 3: Web and Multi-Location Support

Potential capabilities: web-based inventory management, multiple store locations, separate inventory quantities by location, location-aware stock management, and broader business inventory oversight.

These capabilities are future possibilities, not confirmed MVP requirements.

13. Final Product Definition

Smart Stock is a mobile-first inventory management application for small businesses that need a simple way to track products and maintain accurate stock records.

The initial version focuses on a single convenience store with one user and approximately 200 products. Users can register products manually, search inventory, record incoming and outgoing stock, conduct physical stock counts, correct discrepancies, review inventory history, and identify low-stock or out-of-stock products.

The MVP also includes a lightweight checkout workflow. Users select products, enter quantities, and complete a sale. The system then deducts the sold quantities from inventory. Actual payment processing is not required.

Barcode or QR scanning is optional. The application must remain usable for products without codes.

The central principle is that inventory accuracy requires both transaction recording and physical verification. Checkout helps record sales, while physical counting provides a way to correct discrepancies when transactions are missed.

The first version prioritizes a reliable, practical inventory workflow over advanced features. Team collaboration, web support, and multiple store locations remain potential future expansions.

> ***Product direction:*** *Start with one store, solve its inventory problems well, and expand only after the core experience is validated.*

14. Pending Team Discussions

Open decisions the team still needs to settle. Each question lists its options; mark the chosen one under **Decision**.

1. Target audience and industry

1.1 Which industry are we targeting first? Choose one primary market rather than trying to serve every business type.

- ☐ Small convenience stores
- ☐ Food and beverage shops
- ☐ General small retail
- ☐ Construction and building materials
- ☐ Clothing and accessories
- ☐ Wearables and electronics
- ☐ All small businesses with physical inventory

1.2 Which geographical market are we designing for? Our current research focus is Myanmar. Confirm whether the MVP is specifically for Myanmar-based businesses.

- ☐ Myanmar only for MVP
- ☐ Myanmar first, expandable later
- ☐ Multiple countries from launch

1.3 What business size are we targeting? Product complexity changes significantly with inventory size and operational scale.

- ☐ Owner-operated microbusiness
- ☐ Small shop with a few employees
- ☐ Small business with multiple branches

1.4 Who will use the product daily? The primary user determines workflows and permissions.

- ☐ Owner only
- ☐ Owner and family members
- ☐ Owner and employees

2. Customer problems and value

2.1 What is the main problem we want to solve? Pick the most important problem to validate through interviews.

- ☐ Inaccurate stock counts
- ☐ Forgotten sales and stock movements
- ☐ Running out of products
- ☐ Time-consuming manual inventory
- ☐ Difficulty managing purchasing and suppliers

- ☐ Lack of visibility into inventory

2.2 What is the current inventory method of our target users? This must be investigated rather than assumed.

- ☐ Paper and notebooks
- ☐ Memory and manual counting
- ☐ Spreadsheets
- ☐ Existing inventory or POS applications
- ☐ A mixture of methods

2.3 What is our main competitive advantage? Existing products already offer many standard inventory features.

- ☐ Simpler user experience
- ☐ Designed for Myanmar shop workflows
- ☐ Works with unreliable connectivity
- ☐ Affordable for small shops
- ☐ Faster stock counting
- ☐ Not yet decided; validate through research

3. Product scope and MVP

3.1 Which inventory workflows are essential for the MVP? Each added workflow affects the five-day development timeline.

- ☐ Product catalog and stock quantities
- ☐ Stock In and Stock Out
- ☐ Physical stock count and adjustments
- ☐ Sales checkout and automatic stock deduction
- ☐ Low-stock alerts
- ☐ Inventory history

3.2 Will Smart Stock include checkout? A checkout workflow is not the same as a full POS system.

- ☐ Yes, lightweight sales recording only
- ☐ Yes, a more complete POS
- ☐ No, inventory management only
- ☐ Decide after user research

3.3 Do we need barcode or QR scanning in the MVP? Scanning may help some retailers but should not prevent manual entry.

- ☐ Manual entry and search only
- ☐ Optional scanning if time permits
- ☐ Required scanning

3.4 How will users record products? Different industries use different product attributes and units.

- ☐ Simple product name and quantity
- ☐ Products with variants such as size or color
- ☐ Products with different measurement units
- ☐ Industry-specific product information

4. Inventory rules and edge cases

4.1 How should we handle different product types and units? For example, drinks by bottle or carton, construction materials by bag or meter, and clothing by size.

- ☐ One basic unit per product
- ☐ Support multiple units and conversions
- ☐ Support product variants
- ☐ Keep it simple for one chosen industry

4.2 Should we track expiry dates and batches? Potentially important for food, beverages, medicine, and other perishable goods.

- ☐ Not in MVP
- ☐ Expiry dates only
- ☐ Batch and expiry tracking
- ☐ Decide based on target industry

4.3 How should low-stock thresholds work? The threshold could be configured per product or use a simpler default.

- ☐ Owner sets a threshold per product
- ☐ One default threshold
- ☐ No alerts in MVP

4.4 Should inventory changes have a reason and audit history? Helps explain discrepancies and distinguish sales, damage, receiving, and corrections.

- ☐ Record basic movement history
- ☐ Require a reason for adjustments
- ☐ Detailed audit history
- ☐ Defer advanced history features

5. Technical and operational decisions (High · 0 of 4 discussed)

5.1 What happens when internet access is unavailable? Offline support can significantly affect architecture and testing.

- ☐ Internet required
- ☐ Allow essential offline operations
- ☐ Fully offline-first with synchronization

5.3 How will users add products initially? Initial setup effort may determine whether owners adopt the product.

- ☐ Manual entry
- ☐ Import from spreadsheet
- ☐ Bulk entry or scanning
- ☐ Manual entry for MVP; import later

5.4 Will the MVP require login and cloud synchronization? This affects security, setup, and implementation effort.

- ☐ Local prototype only
- ☐ Single account with cloud storage
- ☐ Login plus multi-device synchronization
- ☐ Decide with the development team

6. Success and business model

6.1 How will we measure whether the MVP is successful? Define a measurable outcome before implementation.

- ☐ Users complete stock updates successfully
- ☐ Fewer stock discrepancies
- ☐ Less time spent counting stock
- ☐ Users continue using it after testing
- ☐ Successful demo of core workflows

6.2 What is the business model? It may not need to be implemented now, but it can influence product assumptions.

- ☐ Free tool
- ☐ Freemium (I would like to choose this one: Liam)
- ☐ Monthly subscription
- ☐ Not decided